Homework 1: Image Enhancement

Computer Vision

Fall 2026

In this homework, you will build and compare a small pipeline for enhancing degraded images using classical image processing methods. The goal is not to perfectly reconstruct the original scene, but to justify your choices, observe the effects of the methods, and evaluate trade-offs among detail visibility, noise, and natural appearance.

Assignment

Build and document a pipeline to enhance images affected by non-uniform illumination, low contrast, and possible noise. Apply the methods to a small set of images and compare the alternatives quantitatively and qualitatively.

Data

Use 3 to 5 color images of your choice that exhibit at least one of the following conditions: low contrast, shadows/non-uniform illumination, or poorly visible details. You may use your own photographs or images with a compatible license; cite the source and license. The images must be included in the repository/submission. **Do not use synthetic images as your only evaluation set!**

To make the comparison reproducible, also create a degraded version of each original image by reducing its contrast and adding a small amount of Gaussian noise. Keep both the original and degraded versions. In your report, clearly state that the original images serve as references for evaluating the synthetic degradations, and that a clean reference may not be available in real-world scenarios.

Methods to implement and compare

Implement, apply, and compare at least the following three approaches, keeping the data and evaluation parameters fixed:

1. **Baseline:** the degraded image without processing.
2. **Linear contrast stretching:** rescale the intensity levels. Specify whether you operate per channel or on luminance; discuss any resulting color shifts.
3. **Histogram equalization:** use an appropriate representation for color images, for example, equalizing the luminance channel in $CIE-Lab$ or the V channel in HSV. Comment on possible color changes and noise amplification.

Add a **fourth method of your choice** from among: gamma correction, a simplified homomorphic filter, classical denoising (for example, OpenCV's bilateral filter or non-local means), or a motivated combination of the methods above. If you choose denoising, explain that noise reduction may also smooth details.

Evaluation

For each image and method, produce a side-by-side visual comparison (original, degraded, result). For the synthetically degraded images, compute at least two metrics, including the following mandatory metric:

- **PSNR** with respect to the original;
- one additional metric of your choice: SSIM (using a non-deep-learning library such as `scikit-image`) or a simple metric that you implement yourself (for example, mean absolute error or mean squared error).

Report the metrics in a table and discuss at least one case in which a numerical improvement does not necessarily correspond to a more visually pleasing result.

Also include a short qualitative analysis: for at least two images, state which method you prefer, which details became easier to see, and which artifacts or trade-offs you observed (noise, halos, unnatural colors, loss of detail).

What to submit

Submit a single PDF report (2–4 pages, excluding references), including:

- objective, images, and sources/licenses;
- a concise description of the methods and parameters;
- clear comparative figures;
- a table of metrics for each image and method;
- a discussion of results and trade-offs, including at least two analyzed cases.
- a link to an accessible repository containing:
  1. **Python source code**, organized and commented sufficiently to be understandable.
  2. **Original and degraded images** used in the experiments, or precise instructions and references for obtaining them if the license does not allow redistribution.
  3. **Generated results**: processed images and a CSV file or equivalent containing the metrics.
  4. **README** with the Python version, dependencies, command to run the experiment, and file structure.

Minimum reproducibility requirements

The command documented in the README must run the complete pipeline from the input images and produce results and metrics. Set a fixed random seed for noise generation and document the degradation parameters. Avoid unnecessary dependencies; Python, OpenCV, and NumPy are sufficient, along with an SSIM library if you choose to use one.

Grading criteria

Criterion	Weight
Correctness and completeness of the method implementations	30%
Quality and reproducibility of the experiment and metrics	20%
Visual comparison and readability of figures/tables	20%
Critical analysis of results, artifacts, and trade-offs	20%
Clarity of the report, code organization, and README	10%

Constraints and suggestions

- Use OpenCV and NumPy as your primary tools. Classical tools for metrics and visualization are allowed.
- Deep learning, neural networks, and pre-trained models are not allowed.
- Keep the images and degradations fixed when comparing methods.
- Do not select a method based on a single metric alone; justify your choice with visual and quantitative evidence.

Pre-submission checklist

The README allows another person to run the pipeline.

Every image and degradation used is identified and reproducible.

All required methods and the additional one are included.

Figures show inputs and results using consistent scales.

The table includes PSNR and at least one additional metric.

The report discusses results, limitations, and observed artifacts.

Image sources and licenses are stated.